

Módulo 2 — Prompt Engineering Profesional

Capítulo 17 — Patrones de Prompt Engineering

"Conocer un patrón no significa aplicarlo siempre. La ingeniería consiste en saber cuándo utilizarlo y cuándo descartarlo."

Objetivos de aprendizaje

- Comprender qué es un patrón de Prompt Engineering.
 - Diferenciar técnicas aisladas de patrones reutilizables.
 - Analizar por qué surgieron los patrones modernos.
 - Introducir un marco para seleccionar el patrón adecuado según el problema.
-

Introducción

En el capítulo anterior tratamos al prompt como un componente de ingeniería. Aprendimos a diseñarlo, estructurarlo, evaluarlo y gestionarlo durante su ciclo de vida.

Sin embargo, no todos los problemas requieren el mismo tipo de prompt.

Solicitar un resumen, clasificar documentos, generar código, planificar tareas o coordinar agentes demanda estrategias diferentes.

Con el tiempo, la comunidad comenzó a identificar soluciones recurrentes para este tipo de situaciones. Esas soluciones reutilizables dieron origen a los **patrones de Prompt Engineering**.

¿Qué es un patrón?

Un patrón describe una forma probada de resolver un problema recurrente dentro de un contexto determinado.

No es una receta universal.

Tampoco constituye una garantía de éxito.

Su valor reside en capturar experiencia acumulada y ofrecer un punto de partida para el diseño.

En Prompt Engineering, un patrón define una manera de estructurar la interacción con el modelo para obtener un comportamiento más consistente.



¿Por qué aparecieron?

Los primeros usuarios de Large Language Models (LLMs) improvisaban cada prompt desde cero.

A medida que creció el uso profesional de estos modelos, comenzaron a repetirse ciertos problemas:

- respuestas inconsistentes;
- razonamientos incompletos;
- dificultad para resolver tareas complejas;
- baja reutilización entre proyectos.

Los patrones surgieron como una respuesta a estas limitaciones.

Clasificación inicial

A lo largo de este capítulo estudiaremos los principales patrones modernos, presentados en el orden en que serán desarrollados:

Patrón	Objetivo principal	Costo relativo
Zero-Shot	Resolver tareas sin ejemplos previos.	Bajo
One-Shot	Guiar el comportamiento mediante un ejemplo.	Bajo
Few-Shot	Generalizar a partir de múltiples ejemplos.	Medio
Chain of Thought	Favorecer el razonamiento paso a paso.	Medio
Self-Consistency	Comparar distintos razonamientos antes de responder.	Alto
ReAct	Combinar razonamiento y acciones.	Variable
Tree of Thoughts	Explorar múltiples caminos de resolución.	Alto

Cada uno responde a problemas diferentes y presenta ventajas y limitaciones particulares. Los patrones de mayor complejidad —Chain of Thought, Self-Consistency y Tree of Thoughts— incrementan el consumo de tokens y la latencia; su adopción debe justificarse por la complejidad o el riesgo de la tarea, no por preferencia técnica.

Caso de estudio

Un equipo desarrolla un asistente para clasificar reclamos de clientes.

Inicialmente utiliza un único prompt genérico.

Tras varias iteraciones descubre que incorporar ejemplos representativos mejora significativamente la precisión de la clasificación.

Sin cambiar el modelo, el equipo adopta un patrón **Few-Shot** y obtiene resultados más estables.

La mejora no provino del modelo, sino del patrón utilizado para interactuar con él.

Buenas prácticas

- Seleccionar el patrón según el problema.
- Comprender el propósito antes de aplicarlo.
- Validar empíricamente los resultados.

- Combinar patrones solo cuando exista una justificación clara.
-

Errores frecuentes

- Aplicar siempre el mismo patrón.
 - Elegir una técnica por moda y no por necesidad.
 - Asumir que los patrones reemplazan un buen diseño del prompt.
 - Evaluar únicamente casos favorables.
-

Ideas clave

- Los patrones capturan experiencia reutilizable.
 - Ningún patrón es universal.
 - La selección del patrón constituye una decisión de ingeniería.
-

Transición hacia la siguiente sección

En la próxima sección estudiaremos el patrón **Zero-Shot Prompting**, analizando cuándo resulta suficiente y cuáles son sus principales limitaciones en soluciones empresariales.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 2 — Prompt Engineering Profesional

Capítulo 17 — Patrones de Prompt Engineering

Sección 02 — Zero-Shot Prompting

"La solución más simple suele ser suficiente... hasta que el problema demuestra lo contrario."

Objetivos de aprendizaje

- Comprender el patrón Zero-Shot Prompting.
- Identificar cuándo resulta adecuado utilizarlo.
- Analizar sus ventajas y limitaciones.
- Establecer criterios para decidir cuándo evolucionar hacia otros patrones.

Introducción

El patrón **Zero-Shot Prompting** representa la forma más directa de interactuar con un Large Language Model (LLM).

Consiste en solicitar una tarea sin proporcionar ejemplos previos.

Aunque pueda parecer el enfoque más básico, continúa siendo uno de los más utilizados en aplicaciones empresariales debido a su simplicidad, bajo costo de mantenimiento y rapidez de implementación.

Sin embargo, la simplicidad no debe confundirse con universalidad.

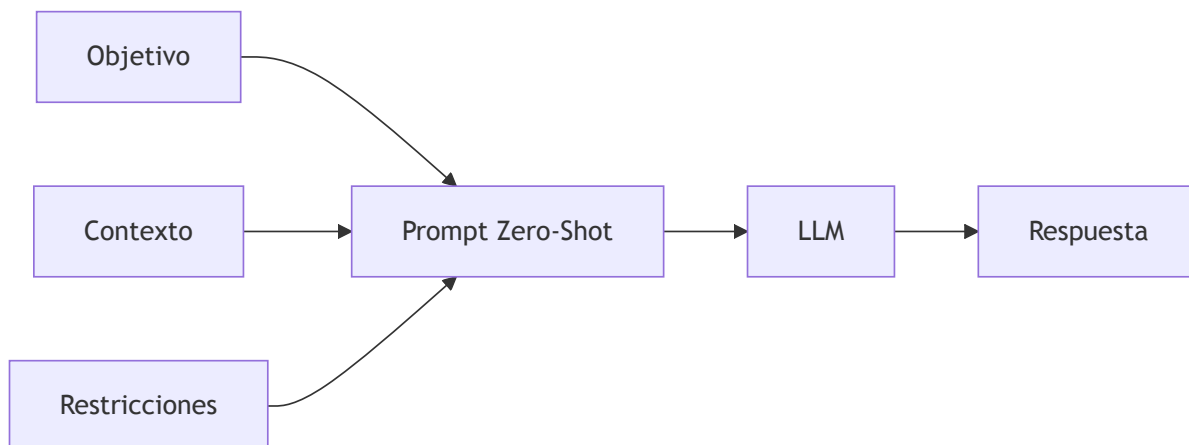
Comprender cuándo Zero-Shot resulta suficiente constituye una competencia fundamental para cualquier AI Engineer.

¿Qué es Zero-Shot Prompting?

En este patrón, el modelo recibe únicamente la descripción de la tarea.

No dispone de ejemplos que indiquen cómo debería responder.

Debe inferir el comportamiento esperado utilizando el conocimiento adquirido durante su entrenamiento y la información disponible en el prompt.



Un prompt Zero-Shot típico para clasificación de texto luce así:

```
Clasificá el siguiente texto en una de estas categorías: queja, consulta, sugerencia.  
  
Texto: "El paquete llegó dañado y nadie responde mis mensajes."
```

El modelo infiere el criterio de clasificación únicamente a partir de las instrucciones, sin ejemplos de referencia.

¿Cuándo utilizar Zero-Shot?

Zero-Shot suele ser apropiado cuando:

- la tarea es bien conocida por el modelo;
- existe poca ambigüedad;
- el formato esperado es sencillo;

- la variabilidad aceptable es elevada;
- la precisión absoluta no constituye un requisito crítico.

Algunos ejemplos incluyen:

- generación de resúmenes;
- traducciones;
- clasificación simple;
- extracción de información básica;
- redacción de contenido.

Una señal práctica para evaluar si Zero-Shot es suficiente: si en una muestra representativa de ejecuciones un porcentaje significativo de respuestas requiere corrección manual, es momento de considerar la incorporación de ejemplos mediante One-Shot o Few-Shot.

Ventajas

Ventaja	Descripción
Simplicidad	Prompts fáciles de construir y mantener.
Bajo costo	Menor consumo de tokens.
Flexibilidad	Adaptación rápida a distintos escenarios.
Rapidez	Desarrollo e iteración más ágiles.

Limitaciones

Cuando aumenta la complejidad del problema, también lo hacen las limitaciones del patrón.

Entre las más frecuentes se encuentran:

- respuestas inconsistentes;
- formatos variables;
- interpretaciones ambiguas;
- dificultad para tareas altamente especializadas;

- menor control sobre el comportamiento del modelo.

Estas limitaciones explican la aparición de patrones como One-Shot y Few-Shot, que estudiaremos en las próximas secciones.

Caso de estudio

Una empresa desarrolla un asistente para resumir informes internos.

El equipo comienza utilizando Zero-Shot debido a que el formato de salida es flexible y los usuarios aceptan pequeñas variaciones entre respuestas.

Meses después, el mismo sistema debe generar informes ejecutivos con una estructura estricta.

Las diferencias observadas entre distintas ejecuciones hacen evidente que Zero-Shot ya no resulta suficiente.

El equipo decide evolucionar hacia un patrón basado en ejemplos.

Buenas prácticas

- Utilizar Zero-Shot como punto de partida.
 - Mantener objetivos claros y específicos.
 - Complementar el prompt con restricciones cuando sea necesario.
 - Medir la calidad antes de incorporar patrones más complejos.
-

Errores frecuentes

- Suponer que Zero-Shot resolverá cualquier problema.
 - Utilizarlo en tareas altamente estructuradas sin evaluación previa.
 - Confundir simplicidad con falta de diseño.
 - Escalar soluciones críticas sin validar su comportamiento.
-

Ideas clave

- Zero-Shot constituye el patrón más simple de Prompt Engineering.
 - Su eficacia depende de la naturaleza del problema.
 - La decisión de evolucionar hacia patrones más complejos debe basarse en evidencia y no en preferencias personales.
-

Transición hacia la siguiente sección

En la próxima sección estudiaremos **One-Shot Prompting**, incorporando un único ejemplo para guiar el comportamiento del modelo y reducir la ambigüedad presente en determinadas tareas.

Módulo 2 — Prompt Engineering Profesional

Capítulo 17 — Patrones de Prompt Engineering

Sección 03 — One-Shot Prompting

"Un único ejemplo bien elegido puede eliminar más ambigüedad que un párrafo completo de instrucciones."

Objetivos de aprendizaje

- Comprender el patrón One-Shot Prompting.
- Analizar cómo un ejemplo condiciona el comportamiento del modelo.
- Identificar escenarios donde One-Shot supera a Zero-Shot.
- Comprender sus ventajas y limitaciones en aplicaciones empresariales.

Introducción

Describir el resultado esperado no siempre es suficiente. El formato puede ser complejo, existir múltiples interpretaciones válidas o requerirse un estilo muy específico.

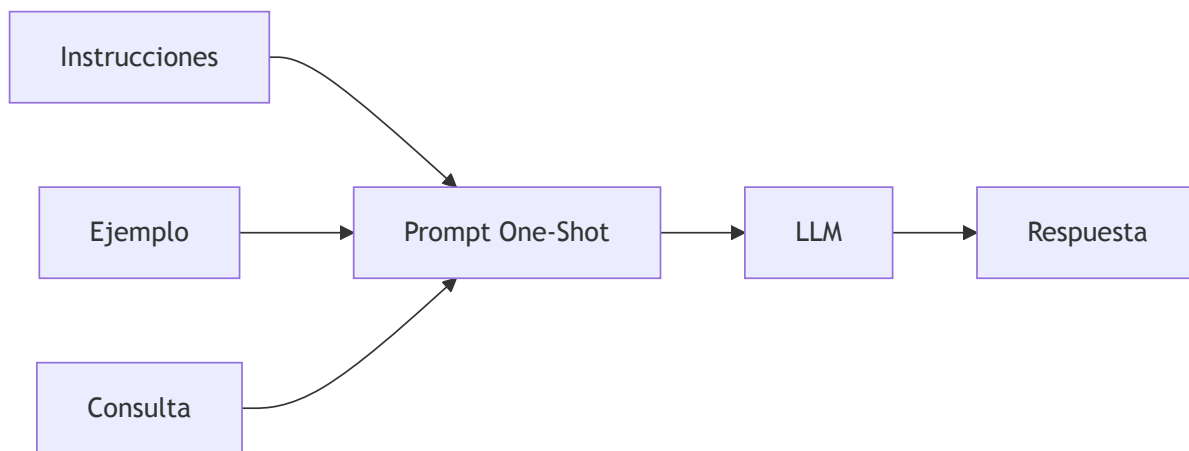
En estos escenarios, proporcionar un único ejemplo representativo reduce significativamente la incertidumbre.

Esta estrategia recibe el nombre de **One-Shot Prompting**.

¿Qué es One-Shot Prompting?

One-Shot consiste en incorporar un único ejemplo completo de entrada y salida dentro del prompt.

Ese ejemplo funciona como una referencia que permite al modelo inferir el patrón esperado antes de resolver el caso real.



El ejemplo no sustituye las instrucciones. Las complementa.

Un prompt One-Shot aplicado a la conversión de reportes técnicos luce así:

```
Convertí el siguiente reporte técnico al formato Markdown corporativo.
```

```
Ejemplo:
```

```
Entrada: "El sistema procesó 1.200 registros el martes."
```

```
Salida:
```

```
## Actividad del sistema
```

```
- Registros procesados: 1.200
```

```
- Fecha: martes
```

```
Ahora convertí este:
```

```
Entrada: "El servidor principal estuvo caído 3 horas el viernes."
```

La estructura es siempre la misma: instrucción, par entrada/salida de referencia, y la consulta real al final.

¿Cuándo utilizar One-Shot?

One-Shot resulta especialmente útil cuando:

Escenario	Beneficio
Formatos específicos	Reduce variaciones de salida.
Clasificaciones simples	Muestra el criterio esperado.
Transformaciones de texto	Establece un patrón claro.
Extracción de datos	Ilustra la estructura objetivo.

En estos casos, un único ejemplo suele ser suficiente para orientar la inferencia.

Ventajas

Entre los principales beneficios se encuentran:

- menor ambigüedad;
- mayor consistencia;
- rápida implementación;
- bajo consumo adicional de tokens;
- facilidad de mantenimiento.

En comparación con Zero-Shot, el costo adicional suele ser reducido y la mejora de calidad puede resultar significativa.

Limitaciones

One-Shot también presenta restricciones.

Un único ejemplo puede:

- representar solo un caso particular;
- introducir sesgos no deseados;
- resultar insuficiente para tareas con alta variabilidad;
- dificultar la generalización cuando existen múltiples excepciones.

En estos escenarios será necesario recurrir a Few-Shot Prompting.

Caso de estudio

Una empresa necesita convertir reportes técnicos en un formato Markdown corporativo.

Con Zero-Shot, la estructura cambia entre documentos.

El equipo incorpora un único ejemplo mostrando exactamente cómo debe verse el resultado.

Sin modificar el modelo, la consistencia aumenta y disminuye el trabajo de edición posterior.

Buenas prácticas

- Seleccionar un ejemplo representativo del caso más frecuente.
 - Mantener coherencia entre el ejemplo y las instrucciones.
 - Evitar ejemplos excesivamente complejos.
 - Actualizar el ejemplo cuando cambien los requisitos.
-

Errores frecuentes

- Utilizar ejemplos que no representan el problema real.
 - Contradecir las instrucciones mediante el ejemplo: si la instrucción pide una lista numerada pero el ejemplo usa viñetas, el modelo priorizará el ejemplo sobre la instrucción.
 - Asumir que un ejemplo cubre todos los escenarios.
 - Elegir ejemplos ambiguos o incompletos.
-

Ideas clave

- One-Shot reduce la ambigüedad mostrando un comportamiento esperado.
 - Un buen ejemplo puede mejorar notablemente la consistencia.
 - La calidad del ejemplo es tan importante como la calidad de las instrucciones.
-

Transición hacia la siguiente sección

En la próxima sección estudiaremos **Few-Shot Prompting**, donde múltiples ejemplos permiten capturar patrones más complejos y mejorar la capacidad de generalización del modelo.

Módulo 2 — Prompt Engineering Profesional

Capítulo 17 — Patrones de Prompt Engineering

Sección 04 — Few-Shot Prompting

"Los ejemplos no enseñan únicamente qué responder. Enseñan cómo pensar el problema."

Objetivos de aprendizaje

- Comprender el patrón **Few-Shot Prompting**.
- Analizar por qué múltiples ejemplos mejoran la generalización.
- Identificar cuándo Few-Shot resulta más apropiado que One-Shot.
- Reconocer sus ventajas, limitaciones y costos.

Introducción

Muchos problemas empresariales presentan una variabilidad considerable. Una única muestra rara vez alcanza para representar todos los escenarios posibles.

En estos casos aparece **Few-Shot Prompting**, un patrón que incorpora varios ejemplos cuidadosamente seleccionados para mostrar al modelo cómo debe comportarse frente a distintas situaciones.

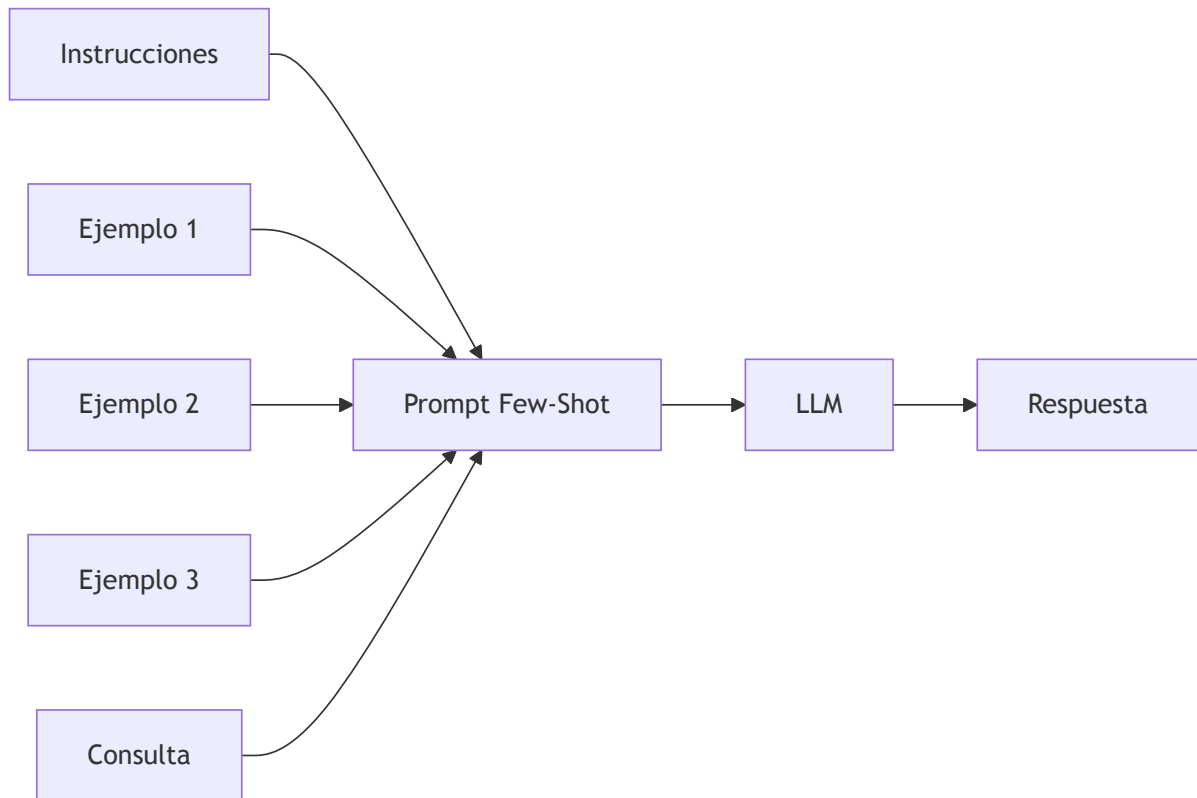
El objetivo ya no consiste únicamente en ilustrar un formato.

Consiste en transmitir un criterio.

¿Qué es Few-Shot Prompting?

Few-Shot Prompting consiste en proporcionar una colección reducida de ejemplos representativos antes de presentar la consulta real.

Cada ejemplo actúa como una evidencia adicional sobre el comportamiento esperado.



Un prompt Few-Shot para clasificación de siniestros luce así:

Clasificá cada siniestro en una de estas categorías: robo, accidente, daño_por_agua, ot

Ejemplo 1:

Entrada: "El cliente reporta que le robaron la billetera en la vía pública."

Salida: robo

Ejemplo 2:

Entrada: "Choque frontal entre dos vehículos en autopista."

Salida: accidente

Ejemplo 3:

Entrada: "Filtración de cañería que dañó el piso de madera del living."

Salida: daño_por_agua

Ejemplo 4:

Entrada: "El cliente encontró su departamento vandalizado tras un viaje."

Salida: robo

Ahora clasificá este:

Entrada: "Granizo destruyó el techo del galpón."

La cantidad de ejemplos no constituye una regla fija. En la práctica, un rango de tres a diez ejemplos suele ser un punto de partida razonable. El número óptimo depende del problema, del modelo y del espacio disponible dentro del Context Window —la ventana de contexto que determina cuánto texto puede procesar el modelo en una sola llamada. Agregar ejemplos más allá del punto de rendimiento marginal decreciente solo incrementa el costo sin mejorar la calidad.

¿Cuándo utilizar Few-Shot?

Few-Shot resulta especialmente útil cuando:

Escenario	Motivo
Clasificaciones complejas	Existen múltiples categorías o excepciones.
Extracción de información	Los documentos presentan estructuras diferentes.
Generación de código	Se requiere mantener convenciones específicas.
Transformación de datos	Existen múltiples formatos de entrada.
Redacción especializada	Es necesario reproducir un estilo consistente.

En estos casos, varios ejemplos permiten al modelo identificar patrones que un único caso difícilmente lograría representar.

Selección de ejemplos

Uno de los errores más frecuentes consiste en incorporar ejemplos de manera aleatoria.

Desde la perspectiva del AI Engineering, los ejemplos deben seleccionarse deliberadamente.

Idealmente deberían cubrir:

- casos típicos;
- casos límite;
- situaciones ambiguas;
- excepciones relevantes;
- formatos diferentes.

El objetivo no es mostrar cantidad, sino diversidad representativa. Elegir únicamente casos exitosos o frecuentes —dejando sin representación los casos límite— es precisamente el tipo de error que Few-Shot debería corregir pero que un conjunto de ejemplos mal seleccionado puede perpetuar.

Costos y beneficios

Few-Shot suele mejorar la calidad de las respuestas, pero también introduce nuevos desafíos.

Beneficio	Impacto
Mayor consistencia	Reduce variaciones entre respuestas.
Mejor generalización	Captura patrones complejos.
Menor ambigüedad	Facilita la interpretación del problema.

Costo	Impacto
Más tokens	Incrementa el costo de Inference (inferencia).
Mayor latencia	El procesamiento puede demorar más.
Mantenimiento	Los ejemplos también deben versionarse.

La decisión de utilizar Few-Shot debe equilibrar estos factores.

Caso de estudio

Una compañía de seguros desarrolla un sistema para clasificar siniestros.

Con Zero-Shot obtiene resultados aceptables.

Con One-Shot mejora la consistencia en los casos más frecuentes.

Sin embargo, los reclamos excepcionales continúan clasificándose incorrectamente.

El equipo incorpora diez ejemplos cuidadosamente seleccionados que representan distintos tipos de siniestros.

La precisión aumenta de forma significativa sin modificar el modelo.

La mejora proviene del diseño del prompt.

Buenas prácticas

- Seleccionar ejemplos que cubran la diversidad del dominio, no solo los casos más frecuentes.

- Evitar redundancias: dos ejemplos que ilustran el mismo escenario aportan menos que uno que cubre un caso límite nuevo.
 - Actualizar los ejemplos cuando cambien las reglas del negocio.
 - Medir el impacto de cada ejemplo incorporado para detectar el punto de rendimiento marginal decreciente.
-

Errores frecuentes

- Utilizar demasiados ejemplos sin aportar información nueva.
 - Elegir únicamente casos exitosos, dejando sin representación los casos límite que son precisamente los que Few-Shot debería resolver.
 - Mezclar criterios inconsistentes entre ejemplos.
 - Ignorar el consumo adicional de tokens al escalar el sistema a producción.
-

Ideas clave

- Few-Shot transmite criterios mediante múltiples ejemplos.
 - La calidad de los ejemplos resulta más importante que su cantidad.
 - Diseñar un conjunto representativo constituye una tarea de ingeniería.
-

Transición hacia la siguiente sección

En la próxima sección estudiaremos **Chain of Thought Prompting**, uno de los patrones que revolucionó la forma de resolver problemas complejos mediante razonamiento paso a paso.

Módulo 2 — Prompt Engineering Profesional

Capítulo 17 — Patrones de Prompt Engineering

Sección 05 — Chain of Thought Prompting

"Resolver un problema complejo rara vez consiste en encontrar la respuesta correcta de inmediato. Consiste en construir un razonamiento que conduzca a ella."

Objetivos de aprendizaje

- Comprender el patrón **Chain of Thought (CoT) Prompting**.
- Analizar por qué el razonamiento paso a paso mejora determinadas tareas.
- Identificar escenarios donde CoT aporta valor y otros donde resulta innecesario.
- Incorporar criterios para utilizar CoT en aplicaciones empresariales.

Introducción

¿Qué sucede cuando el problema no tiene una respuesta directa, sino que requiere construir un camino hacia ella?

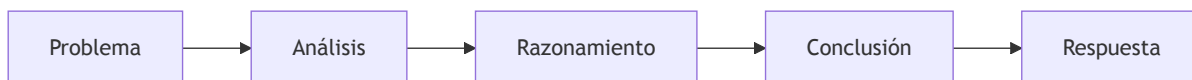
Cálculos, planificación, análisis normativo, diagnóstico, toma de decisiones y resolución de problemas complejos tienen en común que la dificultad no radica únicamente en producir una respuesta, sino en **construir un proceso de razonamiento**.

Para este tipo de situaciones aparece **Chain of Thought Prompting (CoT)**.

¿Qué es Chain of Thought?

Chain of Thought consiste en incentivar al modelo a resolver un problema mediante una secuencia organizada de pasos intermedios antes de producir la respuesta final.

Conceptualmente, el patrón busca transformar un problema complejo en una serie de decisiones más pequeñas.



El objetivo no es hacer la respuesta más extensa, sino favorecer un proceso de inferencia más estructurado.

La forma más sencilla de activar CoT es mediante una instrucción explícita en el prompt:

```
Analizá el siguiente incidente de seguridad. Antes de emitir tu conclusión,
razoná paso a paso: identificá las evidencias disponibles, descartá hipótesis
improbables y justificá cada decisión.
```

```
Incidente: "A las 2:15 AM se detectaron 3.000 intentos de login fallidos
desde una única IP ubicada en un país donde la empresa no opera."
```

Esta variante se conoce como **Zero-Shot CoT**: no requiere ejemplos, solo la instrucción de razonar paso a paso. En la variante **Few-Shot CoT**, se incorporan ejemplos donde la respuesta modelo ya incluye el razonamiento explícito, lo que permite al modelo aprender también el estilo y la profundidad del análisis esperado.

¿Cuándo utilizar CoT?

Este patrón resulta especialmente útil cuando la tarea requiere:

Escenario	Motivo
Resolución de problemas	Existen múltiples pasos intermedios.
Diagnósticos	Deben evaluarse distintas evidencias.
Planificación	Es necesario ordenar acciones.
Análisis normativo	Deben justificarse conclusiones.
Arquitectura	Hay que comparar alternativas antes de decidir.

Por el contrario, tareas simples como traducciones o clasificaciones básicas rara vez justifican el costo adicional.

Beneficios y limitaciones

Beneficios

- Favorece respuestas más consistentes.
- Reduce errores en problemas complejos.
- Facilita la justificación de decisiones.
- Hace más visible el proceso seguido por el modelo.

Limitaciones

- Incrementa el consumo de tokens.
- Puede aumentar la latencia.
- No garantiza una conclusión correcta si el razonamiento parte de premisas incorrectas.
- No todas las tareas requieren razonamiento explícito.

Una advertencia importante: los pasos de razonamiento visibles que genera CoT son outputs del modelo, no una ventana directa a su proceso interno de inferencia. Un razonamiento "bien estructurado" puede ser plausible y aun así incorrecto. Los pasos intermedios deben validarse con la misma rigurosidad que cualquier otra salida del sistema.

Caso de estudio

Un equipo desarrolla un asistente para apoyar el análisis de incidentes de ciberseguridad.

Con un enfoque Zero-Shot el modelo suele responder con una causa probable, pero omite justificarla.

Tras incorporar una estrategia basada en Chain of Thought, el sistema comienza a analizar evidencias, descartar hipótesis y construir una explicación antes de emitir la conclusión.

El resultado no solo mejora la precisión percibida, sino también la confianza de los analistas que utilizan la herramienta.

Buenas prácticas

- Reservar CoT para problemas que realmente requieren razonamiento.
- Evaluar el costo adicional de inferencia antes de adoptar el patrón.
- Solicitar pasos claros y verificables, no explicaciones extensas.
- Comparar la calidad de las conclusiones con y sin CoT en una muestra del dominio antes de generalizar a producción.

Errores frecuentes

- Aplicar CoT a tareas triviales donde el costo no se justifica.
- Asumir que más razonamiento implica mejores respuestas.
- Confundir explicaciones largas con razonamiento de calidad.
- No validar las conclusiones obtenidas: los pasos visibles no garantizan que la inferencia sea correcta.

Ideas clave

- Chain of Thought guía el proceso de razonamiento, no solo la respuesta.
- Su mayor valor aparece en problemas complejos donde los pasos intermedios son necesarios para llegar a la conclusión.
- Debe utilizarse cuando el beneficio supera el costo operativo.

Transición hacia la siguiente sección

En la próxima sección estudiaremos **Self-Consistency**, un patrón que amplía Chain of Thought generando múltiples razonamientos alternativos antes de seleccionar la respuesta más consistente.

Módulo 2 — Prompt Engineering Profesional

Capítulo 17 — Patrones de Prompt Engineering

Sección 06 — Self-Consistency

"Cuando un único razonamiento no ofrece suficiente confianza, la ingeniería busca evidencia adicional antes de decidir."

Objetivos de aprendizaje

- Comprender el patrón **Self-Consistency**.
- Analizar cómo múltiples cadenas de razonamiento pueden mejorar la confiabilidad.
- Evaluar el costo y el beneficio de este enfoque.
- Identificar escenarios donde resulta apropiado utilizarlo.

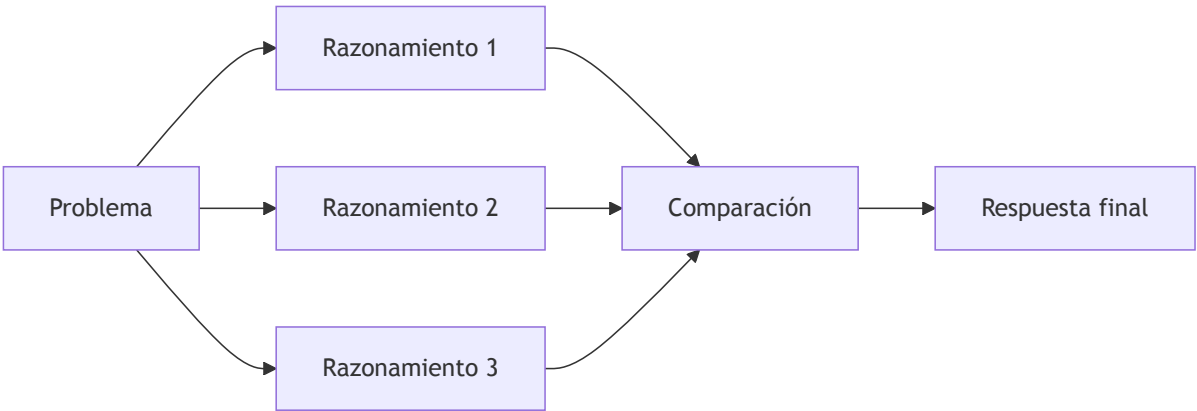
Introducción

Un único razonamiento puede conducir a una conclusión incorrecta. Una premisa equivocada al inicio de una cadena de pensamiento puede propagarse hasta la respuesta final sin que el sistema detecte el error.

El patrón **Self-Consistency** aborda este problema desde un ángulo distinto: en lugar de refinar un único razonamiento, el sistema genera varios razonamientos independientes y selecciona la conclusión que emerge con mayor consistencia entre ellos.

¿Qué es Self-Consistency?

En lugar de confiar en una única secuencia de razonamiento, el sistema explora múltiples caminos posibles y compara sus resultados.



El objetivo no consiste en obtener muchas respuestas, sino en aumentar la probabilidad de seleccionar la más robusta.

¿Cómo se implementa en la práctica? El mecanismo más habitual consiste en realizar múltiples llamadas independientes al LLM con una temperatura mayor a cero —el parámetro que controla la variabilidad de las respuestas— para obtener diferentes caminos de razonamiento sobre el mismo problema. Las conclusiones de cada cadena se extraen y se selecciona la que aparece con mayor frecuencia, un proceso conocido como votación por mayoría. Esta selección puede ejecutarla código externo o un LLM evaluador. Esto implica N llamadas a la API en lugar de una, con el consiguiente impacto multiplicativo en costo y latencia.

¿Cuándo utilizarlo?

Self-Consistency aporta valor cuando:

Escenario	Beneficio
Diagnóstico	Reduce el riesgo de conclusiones apresuradas.
Planificación	Compara estrategias alternativas.
Resolución de problemas complejos	Disminuye errores de razonamiento.
Evaluaciones críticas	Incrementa la confianza antes de decidir.

Para tareas simples, el costo adicional suele superar el beneficio.

Costos operativos

Si Chain of Thought (CoT) ya introduce un costo adicional por el razonamiento explícito, Self-Consistency multiplica ese costo. Al generar N cadenas de razonamiento, el consumo de tokens y el tiempo de inferencia se multiplican aproximadamente por N respecto a un único prompt CoT.

Esto significa que Self-Consistency incrementa:

- el consumo de tokens en proporción al número de cadenas generadas;
- el tiempo de inferencia de forma proporcional;
- el costo económico de cada llamada;
- la complejidad de la orquestación, que ahora incluye lógica de selección y agregación de resultados.

Como referencia práctica, cinco a diez cadenas constituyen un punto de partida razonable para decisiones de riesgo medio; más de diez solo se justifican cuando el impacto de una decisión incorrecta sea significativo. A partir de cierto número, los retornos son decrecientes.

Por ello, debe reservarse para escenarios donde la calidad de la decisión sea más importante que la velocidad de respuesta.

Caso de estudio

Una entidad financiera utiliza un LLM para asistir en la evaluación preliminar de operaciones inusuales.

En lugar de aceptar la primera explicación generada, el sistema produce varias cadenas de razonamiento y compara sus conclusiones.

Cuando existe convergencia entre ellas, aumenta la confianza de la recomendación. Si aparecen diferencias significativas, la operación se deriva a un analista humano.

Buenas prácticas

- Utilizar Self-Consistency únicamente cuando el riesgo lo justifique.
 - Definir criterios objetivos para la selección de la respuesta final: ya sea por mayoría de votos sobre las conclusiones, ya sea por un evaluador externo con criterios formalizados.
 - Registrar métricas de calidad y costo para justificar el uso del patrón.
 - Combinar este patrón con evaluación automatizada que verifique la coherencia de los razonamientos, no solo la frecuencia de las conclusiones.
-

Errores frecuentes

- Aplicarlo indiscriminadamente a cualquier tarea de razonamiento.
 - Confundir cantidad de razonamientos con calidad: más cadenas no garantizan que la respuesta mayoritaria sea correcta.
 - Ignorar el impacto multiplicativo sobre la latencia.
 - No definir cómo seleccionar la respuesta final antes de implementar el patrón.
-

Ideas clave

- Self-Consistency incrementa la robustez comparando múltiples razonamientos independientes.
 - Su principal ventaja aparece en decisiones de alto impacto donde un error de razonamiento tiene consecuencias significativas.
 - El aumento de calidad debe compensar el costo adicional de inferencia, que se multiplica por el número de cadenas generadas.
-

Transición hacia la siguiente sección

En la próxima sección estudiaremos **ReAct (Reason + Act)**, un patrón que combina razonamiento con la capacidad de utilizar herramientas externas para resolver problemas que exceden el conocimiento interno del modelo.

Módulo 2 — Prompt Engineering Profesional

Capítulo 17 — Patrones de Prompt Engineering

Sección 07 — ReAct (Reason + Act)

"Un modelo deja de ser únicamente un generador de texto cuando puede razonar y actuar sobre el mundo que lo rodea."

Objetivos de aprendizaje

- Comprender el patrón **ReAct (Reason + Act)**.
- Analizar cómo combina razonamiento con ejecución de acciones.
- Diferenciar ReAct de Chain of Thought.
- Identificar escenarios empresariales donde aporta ventajas.

Introducción

Los patrones analizados hasta el momento centran su atención en mejorar el proceso de razonamiento del modelo.

Sin embargo, muchas aplicaciones empresariales requieren algo más que producir una buena respuesta. Necesitan consultar información actualizada, ejecutar herramientas, invocar APIs o interactuar con otros sistemas.

Para resolver este tipo de problemas surge **ReAct (Reason + Act)**, un patrón que alterna ciclos de razonamiento con acciones concretas.

¿Qué es ReAct?

ReAct propone que el modelo no resuelva el problema únicamente con su conocimiento interno.

En cambio, puede razonar, decidir qué información necesita, ejecutar una acción y continuar razonando utilizando el resultado obtenido.



Este ciclo puede repetirse tantas veces como resulte necesario.

Es importante distinguir ReAct del mecanismo de **Tool Calling**: Tool Calling es la interfaz técnica que permite al modelo invocar funciones externas; ReAct es el patrón que decide cuándo y por qué invocarlas y cómo integrar el resultado en el razonamiento. Tool Calling es el mecanismo de ejecución; ReAct es la estrategia de razonamiento y decisión que lo gobierna.

Componentes del patrón

Componente	Función
Reason	Analizar el estado actual del problema.
Act	Invocar una herramienta o ejecutar una acción.
Observe	Incorporar el resultado obtenido.
Decide	Determinar el siguiente paso.

Esta alternancia convierte al modelo en un orquestador capaz de interactuar con su entorno.

¿Cuándo utilizar ReAct?

ReAct resulta especialmente adecuado cuando:

- la respuesta depende de información dinámica;
- es necesario consultar bases de datos;
- deben utilizarse herramientas externas;

- la aplicación integra múltiples servicios;
- el modelo necesita verificar información antes de responder.

En estos escenarios, limitarse al conocimiento del modelo suele ser insuficiente.

Caso de estudio

Un asistente corporativo recibe la consulta:

"¿Cuál es el saldo pendiente del cliente 45821 y qué facturas vencen esta semana?"

Un enfoque tradicional obligaría al modelo a responder con conocimiento incompleto o a rechazar la consulta.

Con ReAct, el sistema:

1. analiza la solicitud;
2. identifica la necesidad de consultar el ERP;
3. ejecuta la herramienta correspondiente;
4. recibe los datos;
5. construye la respuesta utilizando información actualizada.

El valor del patrón reside precisamente en esa capacidad de combinar razonamiento con acciones verificables.

ReAct y la arquitectura de agentes

La afirmación de que ReAct "constituye uno de los fundamentos conceptuales de los agentes modernos" merece una explicación concreta.

En una arquitectura agéntica, el ciclo Reason-Act-Observe-Decide se convierte en el bucle principal de ejecución del agente. Cada iteración del ciclo representa un paso de razonamiento seguido de una acción sobre el entorno. El orquestador —ya sea un framework externo, código propio o una plataforma de agentes— controla cuándo inicia y cuándo termina ese bucle, gestiona el estado entre ciclos y maneja los errores de ejecución.

Frameworks como LangChain o LlamaIndex implementan variantes de este ciclo como patrón central de sus arquitecturas de agentes. El Capítulo 18 profundizará en cómo diseñar ese orquestador en entornos de producción.

Buenas prácticas

- Definir claramente las herramientas disponibles para el modelo.
 - Establecer un límite máximo de iteraciones del ciclo para evitar bucles infinitos con alto costo de tokens.
 - Definir una condición de terminación explícita: ¿cuándo considera el sistema que tiene información suficiente para responder?
 - Registrar cada interacción para auditoría.
 - Diseñar mecanismos de recuperación ante errores en la ejecución de herramientas.
-

Errores frecuentes

- Permitir acciones sin control ni límite de iteraciones.
 - Exponer herramientas innecesarias, ampliando la superficie de fallo del sistema.
 - Asumir que el resultado de una acción es correcto sin validarlo antes de incorporarlo al siguiente paso de razonamiento.
 - Confundir ReAct con Tool Calling: Tool Calling es un mecanismo de ejecución; ReAct es un patrón de razonamiento y decisión que puede usar Tool Calling como uno de sus componentes.
-

Ideas clave

- ReAct integra razonamiento y ejecución en un ciclo iterativo.
 - Permite resolver problemas que exceden el conocimiento interno del modelo.
 - Constituye uno de los fundamentos conceptuales de los agentes modernos: su ciclo Reason-Act-Observe-Decide es el bucle de ejecución central en arquitecturas agénticas.
-

Transición hacia la siguiente sección

En la próxima sección estudiaremos **Tree of Thoughts**, un patrón que amplía la capacidad de razonamiento explorando múltiples caminos de resolución antes de seleccionar la alternativa más prometedora.

Módulo 2 — Prompt Engineering Profesional

Capítulo 17 — Patrones de Prompt Engineering

Sección 08 — Tree of Thoughts

"Las mejores decisiones rara vez aparecen al explorar un único camino. Surgen después de comparar alternativas."

Objetivos de aprendizaje

- Comprender el patrón **Tree of Thoughts (ToT)**.
- Analizar cómo amplía las capacidades de razonamiento de un LLM.
- Diferenciar Tree of Thoughts de Chain of Thought.
- Identificar escenarios donde explorar múltiples alternativas aporta valor.

Introducción

El razonamiento lineal tiene una limitación estructural: es irreversible. Si en Chain of Thought (CoT) el razonamiento inicial toma una dirección incorrecta, toda la conclusión puede verse afectada, y el sistema no puede retroceder a un punto anterior para explorar una alternativa diferente.

Tree of Thoughts (ToT) resuelve esta limitación organizando el razonamiento como un árbol en lugar de una línea. Esto permite al sistema retroceder a cualquier nodo del árbol y continuar por una rama diferente, algo que CoT no puede hacer por su naturaleza secuencial.

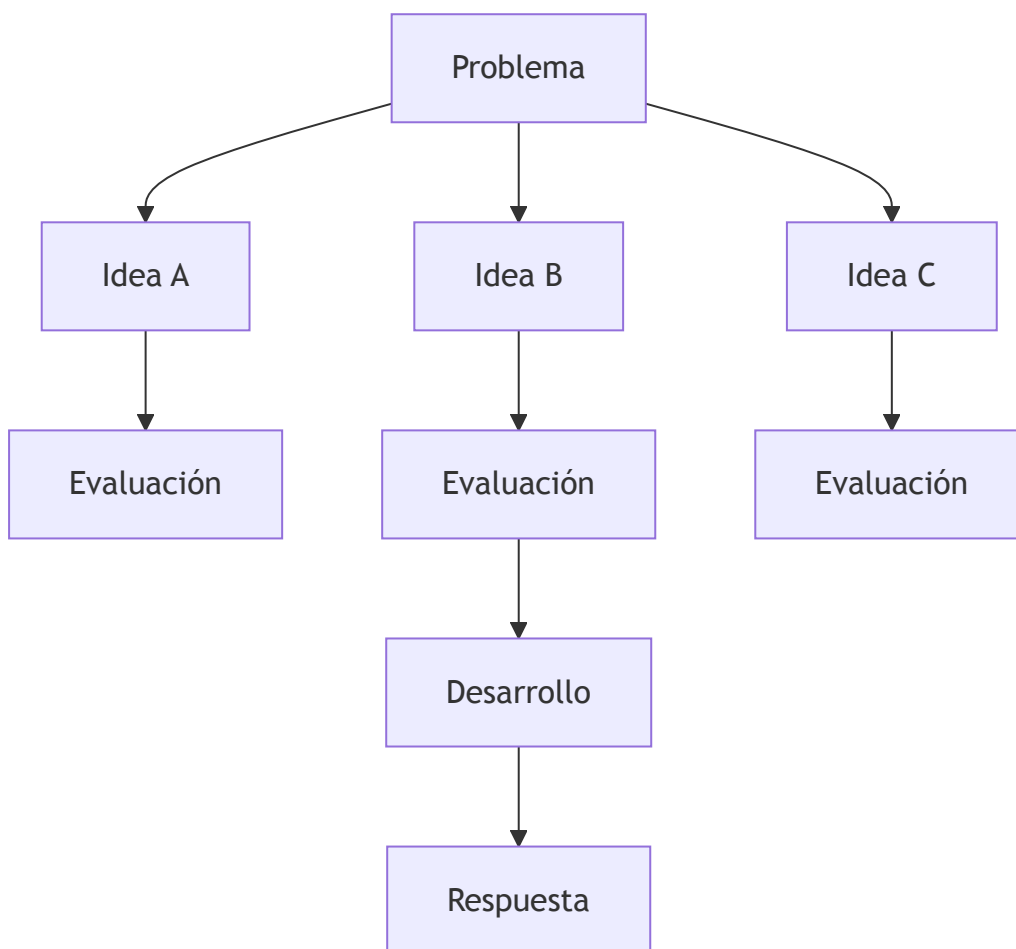
Desde la perspectiva del AI Engineering, este patrón se asemeja más a un algoritmo de búsqueda que a una conversación tradicional.

¿Qué es Tree of Thoughts?

Tree of Thoughts organiza el proceso de razonamiento como un árbol.

Cada nodo representa una posible línea de pensamiento.

El sistema puede expandir, comparar, descartar o profundizar cada rama antes de seleccionar la solución final.



El objetivo no consiste en generar más texto, sino en explorar el espacio de soluciones de forma controlada.

¿Cómo funciona la evaluación y poda de ramas? Este es el mecanismo central del patrón. Cada rama generada debe evaluarse antes de decidir si vale la pena desarrollarla. Existen dos estrategias principales:

- **LLM-as-judge:** el mismo modelo actúa como evaluador de sus propias ramas mediante una llamada separada con un prompt específico. Esta estrategia es flexible pero puede

introducir sesgo de confirmación si el prompt evaluador no se diseña con cuidado.

- **Evaluador externo:** una función con reglas formalizadas de negocio evalúa cada rama. Es más confiable cuando los criterios son objetivos y medibles.

El criterio de poda puede ser un umbral de puntuación mínimo o la selección de las K ramas mejor evaluadas en cada nivel —un mecanismo análogo al beam search en búsqueda heurística. Cada nivel del árbol implica múltiples llamadas a la API, con un impacto de costo proporcional al número de ramas y niveles explorados.

Comparación con Chain of Thought

Aspecto	Chain of Thought	Tree of Thoughts
Caminos explorados	Uno	Múltiples
Evaluación intermedia	Limitada	Explícita
Capacidad de corrección	Baja	Alta
Costo computacional	Menor	Mayor

ToT incrementa el costo de Inference (inferencia), pero puede producir mejores resultados cuando existen numerosas alternativas posibles y el problema permite retroceder y reorientar el razonamiento.

¿Cuándo utilizar Tree of Thoughts?

Este patrón resulta especialmente útil en tareas como:

- planificación estratégica;
- diseño de arquitecturas;
- resolución de problemas abiertos;
- optimización de procesos;
- análisis de múltiples escenarios;
- apoyo a la toma de decisiones.

En problemas simples, el beneficio suele ser inferior al costo adicional.

Caso de estudio

Un equipo debe diseñar la arquitectura de una plataforma empresarial de IA.

No existe una única solución correcta.

El sistema genera tres alternativas:

- arquitectura completamente en la nube;
- infraestructura híbrida;
- despliegue on-premise.

Cada alternativa se evalúa según costo, escalabilidad, seguridad y mantenimiento.

En lugar de responder inmediatamente, el modelo compara las opciones y desarrolla únicamente la que mejor satisface los requisitos del negocio.

El patrón no reemplaza el criterio del arquitecto, pero amplía la capacidad para analizar opciones antes de decidir.

Buenas prácticas

- Definir criterios claros de evaluación antes de iniciar la exploración.
 - Usar un prompt evaluador separado del prompt generador para reducir el sesgo de confirmación en la evaluación.
 - Limitar la cantidad de ramas exploradas y la profundidad del árbol para controlar el costo de inferencia.
 - Registrar las ramas descartadas y sus puntuaciones para poder auditar por qué el sistema eligió una alternativa sobre otra.
-

Errores frecuentes

- Explorar demasiadas alternativas sin un criterio de selección definido.
- Usar el mismo LLM generador como evaluador sin un prompt separado, lo que introduce sesgo de confirmación.

- Utilizar ToT en problemas triviales donde CoT sería suficiente.
 - No registrar las ramas descartadas, lo que impide auditar las decisiones del sistema.
-

Ideas clave

- Tree of Thoughts explora múltiples caminos antes de decidir, con capacidad de retroceder y reorientar el razonamiento.
 - Su diferencia estructural con CoT es la reversibilidad: ToT puede abandonar una rama y continuar por otra.
 - Debe reservarse para escenarios donde el beneficio de explorar alternativas justifique el costo multiplicativo de inferencia.
-

Transición hacia la siguiente sección

En la próxima sección compararemos los patrones estudiados hasta el momento y construiremos un marco de decisión que permita seleccionar el enfoque más adecuado según el problema de negocio.

Módulo 2 — Prompt Engineering Profesional

Capítulo 17 — Patrones de Prompt Engineering

Sección 09 — Marco de decisión y comparación general

"Conocer muchos patrones aporta poco valor si no se sabe cuál utilizar frente a un problema concreto."

Objetivos de aprendizaje

- Comparar los principales patrones de Prompt Engineering.
- Construir un marco de decisión para seleccionar el patrón adecuado.
- Relacionar la complejidad del problema con la estrategia de prompting.
- Introducir criterios de ingeniería para la toma de decisiones.

Introducción

A lo largo de este capítulo analizamos siete patrones de Prompt Engineering, cada uno diseñado para resolver un conjunto específico de problemas.

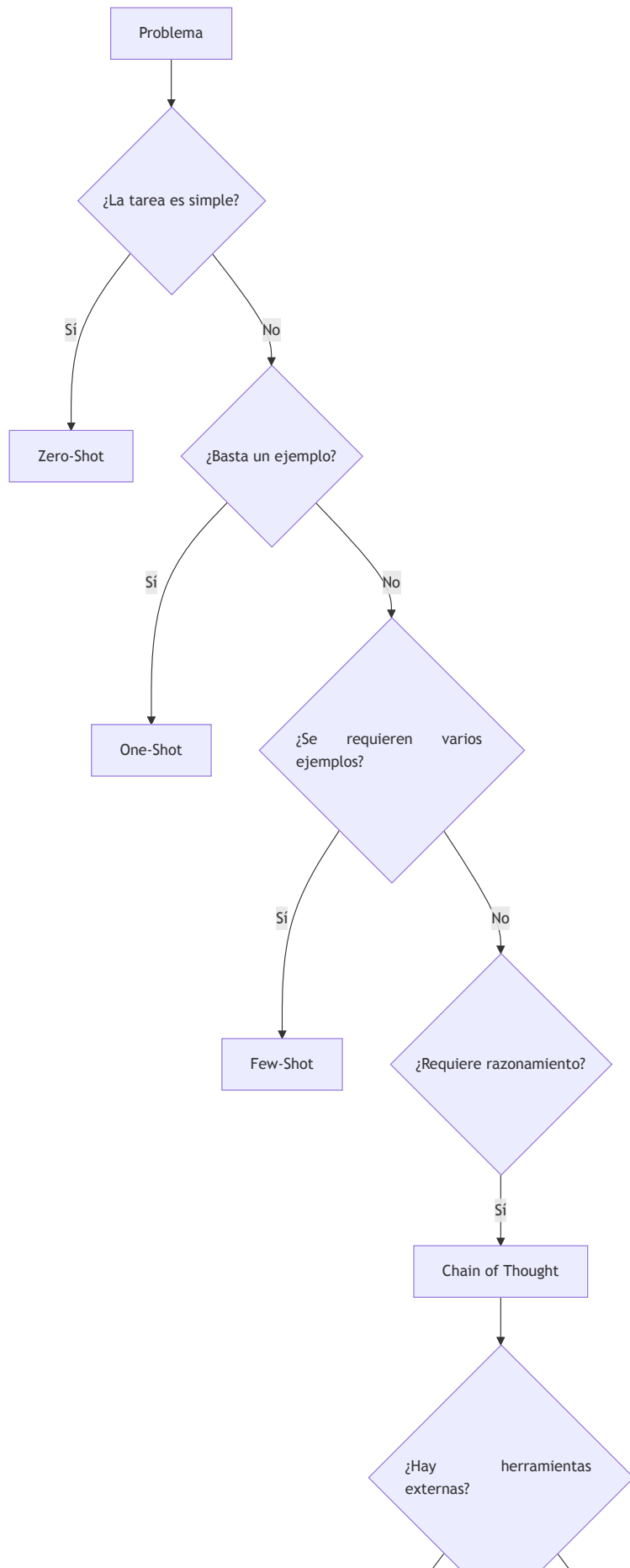
Sin embargo, una pregunta continúa abierta:

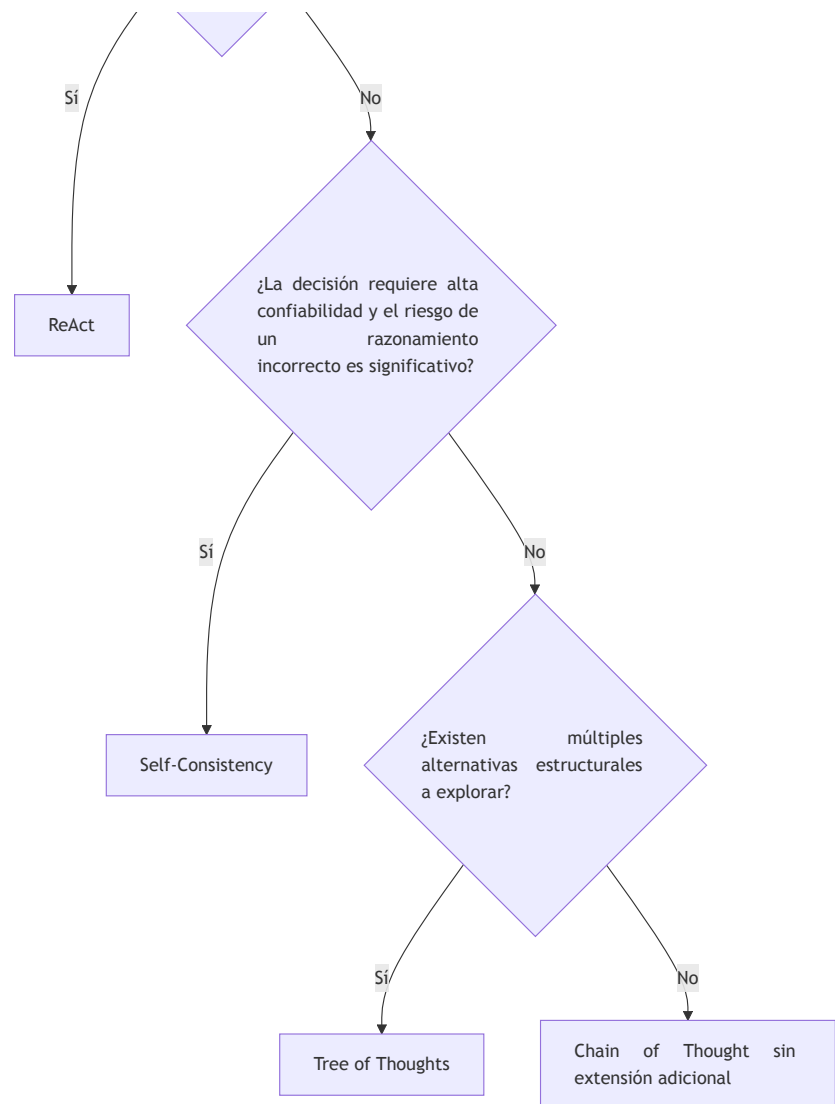
¿Cómo decidir qué patrón utilizar?

La respuesta no depende del modelo ni de las preferencias del ingeniero. Depende del problema que se intenta resolver y de las restricciones del entorno de producción.

Un marco de decisión

La selección de un patrón puede entenderse como un proceso incremental. Se recomienda comenzar con la estrategia más simple capaz de resolver el problema y evolucionar únicamente cuando la evidencia demuestre que es necesario.





Comparación general

Patrón	Complejidad	Costo	Latencia	Caso de uso típico
Zero-Shot	Baja	Bajo	Baja	Tareas simples y bien conocidas
One-Shot	Baja	Bajo	Baja	Formatos específicos con poca variabilidad
Few-Shot	Media	Medio	Media	Generalización con variabilidad moderada
Chain of Thought	Media	Medio	Media	Razonamiento complejo paso a paso
Self-Consistency	Alta	Alto	Alta	Decisiones críticas que requieren alta confiabilidad
ReAct	Alta	Variable	Variable	Problemas que requieren información dinámica o herramientas externas
Tree of Thoughts	Muy alta	Alto	Alta	Exploración de múltiples alternativas estructurales

Ingeniería antes que técnica

Un error frecuente consiste en adoptar el patrón más sofisticado disponible suponiendo que producirá mejores resultados.

En ingeniería ocurre exactamente lo contrario.

La mejor solución suele ser aquella que satisface los requisitos utilizando la menor complejidad posible.

Cada patrón introduce costos adicionales en términos de tokens, latencia, mantenimiento y evaluación. Por ello, la decisión debe justificarse mediante métricas y necesidades concretas del negocio.

Caso de estudio

Una organización desarrolla tres asistentes con el mismo Large Language Model (LLM):

- un **generador de resúmenes**, donde el formato de salida es flexible y la variabilidad entre respuestas es aceptable: emplea Zero-Shot;
- un **analizador de contratos**, que debe identificar cláusulas problemáticas y justificar cada observación: emplea Chain of Thought;
- un **agente que coordina procesos entre varios sistemas**, consultando bases de datos y verificando información en tiempo real antes de responder: emplea ReAct.

La diferencia no reside en la capacidad del modelo, sino en la naturaleza del problema que cada aplicación debe resolver.

Self-Consistency y Tree of Thoughts no aparecen en este caso porque ninguno de los tres asistentes presenta las condiciones que los justifican: no hay decisiones de alta criticidad donde un único razonamiento sea insuficientemente robusto, ni problemas con múltiples alternativas estructurales que requieran exploración comparada. Cuando esas condiciones aparezcan —por ejemplo, en un sistema de evaluación de riesgo crediticio o en un planificador de proyectos con opciones de arquitectura alternativas— serán los patrones indicados.

Buenas prácticas

- Comenzar por el patrón más simple.
- Incrementar la complejidad solo cuando sea necesario y la evidencia lo justifique.
- Medir costo y calidad antes de cambiar de estrategia.
- Documentar la justificación de cada decisión.

Errores frecuentes

- Elegir patrones por tendencia tecnológica o por sofisticación percibida.
- Sobredimensionar la solución frente a la complejidad real del problema.
- Cambiar de patrón sin evidencia que respalde el cambio.

- Ignorar el impacto operativo acumulado de tokens, latencia y mantenimiento.
-

Ideas clave

- No existe un patrón universal.
 - La selección depende del problema y del contexto operativo.
 - La simplicidad constituye una virtud cuando cumple los objetivos del negocio.
-

Transición hacia la siguiente sección

En la próxima sección cerraremos el capítulo analizando cómo combinar varios patrones dentro de una misma solución y prepararemos el paso hacia el Capítulo 18, donde estudiaremos Prompt Engineering para entornos de producción.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."